

RESPUESTAS ABAJO DEL TODO

TEMA 1. Clases y objetos

1. ¿Cuáles son las cuatro características básicas de la programación orientada a objetos? Describe brevemente cada una
2. Cita cuatro lenguajes populares que permitan la programación orientada a objetos
3. Los paradigmas anteriores a la POO, ¿Qué es la **programación estructurada**? y, todavía mejor, ¿Qué es la **programación modular**?
4. ¿Qué tres elementos definen a un objeto en programación orientada a objetos?
5. ¿Qué es una clase? ¿Es lo mismo que un objeto? ¿Qué es una instancia? ¿Todos los lenguajes orientados a objetos manejan el concepto de clase?
6. ¿Dónde se almacenan en memoria los objetos? ¿Es igual en todos los lenguajes? ¿Qué es la **recolección de basura**?
7. ¿Qué es un método? ¿Qué es la **sobrecarga de métodos**?
8. Ejemplo mínimo de clase en Java, que se llame Punto, con dos atributos, x e y, con un método que se llame **calculaDistanciaAOrigen**, que calcule la distancia a la posición 0,0. Por sencillez, los atributos deben tener visibilidad por defecto. Crea además un ejemplo de uso con una instancia y uso del método
9. ¿Cuál es el punto de entrada en un programa en Java? ¿Qué es **static** y para qué vale? ¿Sólo se emplea para ese método **main**? ¿Para qué se combina con **final**?

Respuesta

10. Intenta ejecutar un poco de Java de forma básica, con los comandos **javac** y **java**. ¿Cómo podemos compilar el programa y ejecutarlo desde línea de comandos? ¿Java es compilado? ¿Qué es la **máquina virtual**? ¿Qué es el *byte-code* y los ficheros **.class**?

Respuesta

11. En el código anterior de la clase **Punto** ¿Qué es **new**? ¿Qué es un **constructor**? Pon un ejemplo de constructor en una clase **Empleado** que tenga DNI, nombre y apellidos

Respuesta

12. ¿Qué es la referencia **this**? ¿Se llama igual en todos los lenguajes? Pon un ejemplo del uso de **this** en la clase **Punto**

Respuesta

13. Añade ahora otro nuevo método que se llame **distanciaA**, que reciba un **Punto** como parámetro y calcule la distancia entre **this** y el punto proporcionado

Respuesta

14. El paso del **Punto** como parámetro a un método, es **por copia** o **por referencia**, es decir, si se cambia el valor de algún atributo del punto pasado como parámetro, dichos cambios afectan al objeto fuera del método? ¿Qué ocurre si en vez de un **Punto**, se recibiese un entero (**int**) y dicho entero se modificase dentro de la función?

Respuesta

15. ¿Qué es el método **toString()** en Java? ¿Existe en otros lenguajes? Pon un ejemplo de **toString()** en la clase **Punto** en Java

Respuesta

16. Reflexiona: ¿una clase es como un **struct** en C? ¿Qué le falta al **struct** para ser como una clase y las variables de ese tipo ser instancias?

Respuesta

17. Qitemos un poco de magia a todo esto: ¿Como se podría “emular”, con **struct** en C, la clase **Punto**, con su función para calcular la distancia al origen? ¿Qué ha pasado con **this**?

Respuesta



Fundamento #1: ¿Por qué existe la POO?

Imagina que estás construyendo una casa:

Paradigma	Analogía	Problema que resuelve
Programación procedural (C)	Tienes ladrillos, cemento y herramientas sueltas en el suelo	Código desorganizado, difícil de mantener
POO (Java)	Tienes objetos : Puerta , Ventana , Pared con sus propias propiedades y comportamientos	Código modular, reutilizable y fácil de entender

La POO nace para **modelar el mundo real** en código: los objetos tienen **datos** (atributos) y **comportamientos** (métodos).

Las 4 Pilares de la POO (Pregunta 1)

Pilar	Qué es	Ejemplo real	Ejemplo en código
1. Abstracción	Ocultar complejidad y mostrar solo lo esencial	Un coche: tú usas el volante/acelerador, no necesitas saber cómo funciona el motor	<code>coche.acelerar()</code> → no ves el código interno
2. Encapsulamiento	Proteger los datos internos del objeto	Tu DNI está en tu cartera, no lo muestras a cualquiera	Atributos <code>private</code> + métodos <code>public</code> para acceder
3. Herencia	Reutilizar código creando jerarquías	<code>Perro</code> hereda de <code>Animal</code> → tiene <code>comer()</code> y <code>dormir()</code> sin reescribir	<code>class Perro extends Animal { ... }</code>
4. Polimorfismo	Un mismo comportamiento con diferentes implementaciones	<code>Animal.hablar()</code> : el perro "ladra", el gato "maúlla"	Sobrecarga (<code>overload</code>) y sobrescritura (<code>override</code>)

 **Clave:** Estos 4 pilares trabajan juntos para crear código **mantenible, reutilizable y escalable**.

Lenguajes POO (Pregunta 2)

Lenguaje	Tipo	Notas
Java	Puro POO (casi todo es objeto)	Tu asignatura actual
C++	Híbrido (POO + procedural)	Extensión de C con clases
Python	Multiparadigma (POO + funcional)	Todo es objeto, pero no obligatorio usar POO
C#	Puro POO (Microsoft)	Similar a Java, usado en .NET

 **Curiosidad:** JavaScript es *basado en prototipos*, no en clases (aunque desde ES6 tiene sintaxis de clases).

Programación estructurada vs modular (Pregunta 3)

Programación estructurada (década 1970)

- **Idea:** Organizar código con estructuras de control (`if`, `for`, `while`)
- **Problema:** Código "spaghetti" → difícil de mantener en proyectos grandes
- **Ejemplo en C:**

```
if (edad > 18) {
    printf("Mayor de edad");
}
```

Programación modular (década 1980)

- **Idea:** Dividir el programa en **módulos** (funciones/archivos) reutilizables
- **Avance:** `funcionCalcularIva()` se puede usar en múltiples partes
- **Limitación:** Los datos y funciones están **separados** → no modelan objetos del mundo real

POO (década 1990+)

- **Idea revolucionaria:** **Unir datos y comportamientos** en un mismo contenedor (la clase)
- **Resultado:** `Producto.calcularIva()` → el IVA pertenece al producto, no es una función suelta

🔗 ¿Qué es un objeto? (Pregunta 4)

Un objeto tiene **3 elementos esenciales**:

Elemento	Descripción	Ejemplo: Coche
1. Estado	Datos que lo describen (atributos)	<code>color = "rojo", velocidad = 60</code>
2. Comportamiento	Acciones que puede realizar (métodos)	<code>acelerar(), frenar()</code>
3. Identidad	Referencia única en memoria	Dirección <code>0x7f8b4c3d2a10</code>

💡 **Analogía:** Tu DNI es tu *identidad*, tu nombre/edad son tu *estado*, y "caminar/hablar" son tus *comportamientos*.

🏗️ Clase vs Objeto vs Instancia (Pregunta 5)

Concepto	Definición	Analogía
Clase	Plantilla/blueprint para crear objetos	Plano arquitectónico de una casa
Objeto	Entidad concreta con estado y comportamiento	Una casa construida según el plano
Instancia	Sinónimo de "objeto" (cuando creas un objeto, lo "instancias")	"Esta casa es una instancia del plano"

Ejemplo en Java:

```
// Clase = plantilla
class Coche {
    String color;
    void arrancar() { ... }
```

```
}

// Instanciación = crear objeto
Coche miCoche = new Coche(); // miCoche es un OBJETO/INSTANCIA de la clase Coche
```

! ¿Todos los lenguajes tienen clases?

- **Sí:** Java, C#, C++
- **No:** JavaScript (usa *prototipos*), Go (usa *structs* con métodos)

Memoria y recolección de basura (Pregunta 6)

¿Dónde se almacenan los objetos?

Zona de memoria	Qué almacena	Java	C++
Stack (pila)	Variables primitivas y referencias	<code>int x = 5;</code> <code>Coche c;</code>	Igual
Heap (montón)	Objetos completos	<code>new Coche()</code> → aquí	<code>new Coche()</code> → aquí

Ejemplo visual:

```
Coche miCoche = new Coche();
//  └──┬──┘      └──┬──┘
//      │            │
//      └─┬────────┘ Objeto en HEAP
//      │
//      └─┬────────┘ Referencia en STACK (apunta al heap)
```

Recolección de basura (Garbage Collection)

- **Problema en C++:** Tú debes liberar memoria manualmente (`delete`)
- **Solución en Java:** La JVM detecta objetos sin referencias y los elimina automáticamente
- **Ventaja:** Evita *memory leaks* (fugas de memoria)
- **Desventaja:** Ligero overhead de rendimiento (hoy en día mínimo)

Métodos y sobrecarga (Pregunta 7)

¿Qué es un método?

Un **bloque de código** que define un comportamiento de la clase.

```
class Calculadora {
    // Método que suma dos enteros
    int sumar(int a, int b) {
        return a + b;
    }
}
```

```
    }
}
```

Sobrecarga de métodos (**overloading**)

Mismo nombre, diferentes parámetros → el compilador elige según el contexto.

```
class Calculadora {
    int sumar(int a, int b) {           // Versión 1
        return a + b;
    }

    double sumar(double a, double b) { // Versión 2 (sobrecarga)
        return a + b;
    }

    int sumar(int a, int b, int c) {    // Versión 3 (sobrecarga)
        return a + b + c;
    }
}

// Uso:
Calculadora calc = new Calculadora();
calc.sumar(2, 3);           // → llama a versión 1
calc.sumar(2.5, 3.1);      // → llama a versión 2
calc.sumar(1, 2, 3);       // → llama a versión 3
```

⚠ **No es sobrecarga:** Cambiar solo el tipo de retorno → ERROR de compilación.

🔗 Ejemplo mínimo: Clase **Punto** (Pregunta 8)

```
// Punto.java
class Punto {
    // Atributos con visibilidad por defecto (package-private)
    double x;
    double y;

    // Método que calcula distancia al origen (0,0)
    double calculaDistanciaAOrigen() {
        return Math.sqrt(x * x + y * y);
    }
}

// Main.java (ejemplo de uso)
public class Main {
    public static void main(String[] args) {
        // Crear instancia
        Punto p = new Punto();
    }
}
```

```

        // Asignar valores a atributos
        p.x = 3.0;
        p.y = 4.0;

        // Usar método
        double distancia = p.calculaDistanciaAOrigen();
        System.out.println("Distancia al origen: " + distancia); // → 5.0
    }
}

```

Compilación y ejecución:

```

javac Punto.java Main.java
java Main

```

 **Visibilidad por defecto:** Sin modificador (`public/private`), solo accesible desde el mismo paquete.

Punto de entrada `main` y `static` (Pregunta 9)

¿Por qué `public static void main(String[] args)`?

Palabra clave	Significado
<code>public</code>	Accesible desde fuera de la clase
<code>static</code>	No necesita instancia para ejecutarse (pertenece a la clase, no al objeto)
<code>void</code>	No devuelve ningún valor
<code>main</code>	Nombre reservado por la JVM como punto de entrada
<code>String[] args</code>	Argumentos de línea de comandos

¿Para qué más sirve `static`?

```

class Matematicas {
    // Constante estática (compartida por todas las instancias)
    public static final double PI = 3.14159;

    // Método estático (no necesita instancia)
    public static double areaCirculo(double radio) {
        return PI * radio * radio;
    }
}

```

```
// Uso sin crear instancia:  
double area = Matematicas.areaCirculo(5.0);
```

static + final = Constante

```
public static final int MAX_USUARIOS = 100;  
// - static: compartido por toda la clase  
// - final: valor inmutable (constante)
```

🔑 Compilación y Máquina Virtual (Pregunta 10)

Flujo de ejecución Java:

```
Código fuente (.java)  
    ↓ [javac]  
Bytecode (.class) → ¡PLATAFORMA INDEPENDIENTE!  
    ↓ [java]  
Máquina Virtual Java (JVM)  
    ↓  
Código nativo (Windows/Linux/macOS)
```

¿Java es compilado o interpretado?

- **Ambos:**
 1. **javac** → compila a **bytecode** (no a código máquina nativo)
 2. **java** → la JVM **interpreta** el bytecode (o lo compila JIT a código nativo en runtime)

¿Qué es el bytecode?

- Código intermedio **independiente de la plataforma**
- Archivos **.class** contienen bytecode
- La JVM lo ejecuta en cualquier sistema operativo

💡 **Ventaja clave:** "Write once, run anywhere" → compilas una vez, ejecutas en Windows/Linux/macOS sin recompilar.

🆕 new y constructores (Pregunta 11)

¿Qué hace **new**?

1. Reserva memoria en el **heap** para el objeto
2. Llama al **constructor** para inicializar el objeto
3. Devuelve una **referencia** al objeto creado

Constructor vs Método

Constructor	Método
Mismo nombre que la clase	Nombre arbitrario
Sin tipo de retorno (ni <code>void</code>)	Tiene tipo de retorno
Se ejecuta automáticamente con <code>new</code>	Se llama explícitamente

Ejemplo: Clase `Empleado`

```
class Empleado {
    String dni;
    String nombre;
    String apellidos;

    // Constructor con parámetros
    Empleado(String dni, String nombre, String apellidos) {
        this.dni = dni;
        this.nombre = nombre;
        this.apellidos = apellidos;
    }
}

// Uso:
Empleado emp = new Empleado("12345678A", "Ana", "García");
```

💡 **Constructor por defecto:** Si no defines ninguno, Java crea uno vacío `Empleado() {}` automáticamente. Pero si defines al menos uno, ¡el por defecto desaparece!

🔗 Referencia `this` (Pregunta 12)

¿Qué es `this`?

- **Referencia al objeto actual** dentro de un método/constructor
- Resuelve ambigüedad entre parámetros y atributos

Ejemplo en `Punto`:

```
class Punto {
    double x;
    double y;

    // Constructor con this para diferenciar parámetros de atributos
    Punto(double x, double y) {
        this.x = x; // this.x → atributo | x → parámetro
        this.y = y;
    }
}
```

```

    double calculaDistanciaAOrigem() {
        // this es opcional aquí, pero explícito
        return Math.sqrt(this.x * this.x + this.y * this.y);
    }
}

```

¿Se llama igual en todos los lenguajes?

Lenguaje	Palabra clave
Java, C++, C#	<code>this</code>
Python	<code>self</code> (primer parámetro explícito)
JavaScript	<code>this</code> (pero con comportamiento diferente)



Método `distanciaA` entre puntos (Pregunta 13)

```

class Punto {
    double x;
    double y;

    Punto(double x, double y) {
        this.x = x;
        this.y = y;
    }

    // Distancia a otro punto (usa this implícitamente)
    double distanciaA(Punto otro) {
        double dx = this.x - otro.x;
        double dy = this.y - otro.y;
        return Math.sqrt(dx * dx + dy * dy);
    }

    // Para prueba rápida
    public static void main(String[] args) {
        Punto p1 = new Punto(0, 0);
        Punto p2 = new Punto(3, 4);

        System.out.println("Distancia: " + p1.distanciaA(p2)); // → 5.0
    }
}

```



Paso por valor vs referencia (Pregunta 14)

Regla de oro en Java:

Tipo	Paso	¿Modificaciones afectan fuera?
Primitivos (<code>int</code> , <code>double</code> , <code>boolean</code>)	Por valor	✗ No
Objetos (<code>Punto</code> , <code>String</code> , arrays)	Por referencia	☒ Sí (pero con matices)

Ejemplo práctico:

```
class Punto {
    double x, y;
    Punto(double x, double y) { this.x = x; this.y = y; }
}

public class Test {
    // Modificar atributo de objeto → SÍ afecta fuera
    static void mover(Punto p) {
        p.x = 100; // Modifica el objeto original
    }

    // Reasignar referencia → NO afecta fuera
    static void reasignar(Punto p) {
        p = new Punto(999, 999); // Solo cambia la referencia LOCAL
    }

    // Primitivo → NO afecta fuera
    static void cambiarInt(int n) {
        n = 999; // Solo modifica copia local
    }

    public static void main(String[] args) {
        Punto p = new Punto(1, 2);
        mover(p);
        System.out.println(p.x); // → 100 ☒ ¡Modificado!

        reasignar(p);
        System.out.println(p.x); // → 100 ✗ ¡No cambió!

        int num = 5;
        cambiarInt(num);
        System.out.println(num); // → 5 ✗ ¡No cambió!
    }
}
```

 **Clave:** Java siempre pasa **por valor**, pero para objetos el "valor" es la **referencia** (dirección de memoria). Por eso puedes modificar el objeto, pero no reasignar la referencia original.

Método `toString()` (Pregunta 15)

¿Para qué sirve?

- Devuelve una **representación textual** del objeto
- Se llama automáticamente en `System.out.println(objeto)`

Ejemplo en **Punto**:

```
class Punto {
    double x, y;

    Punto(double x, double y) {
        this.x = x;
        this.y = y;
    }

    @Override // Buena práctica: indica que sobrescribes un método de Object
    public String toString() {
        return "(" + x + ", " + y + ")";
    }

    public static void main(String[] args) {
        Punto p = new Punto(3.5, 4.2);
        System.out.println(p); // → (3.5, 4.2) ¡sin llamar explícitamente a
        toString()!
    }
}
```

¿Existe en otros lenguajes?

Lenguaje	Método equivalente
Python	<code>__str__()</code>
C++	Sobrecargar <code>operator<<</code>
C#	<code>ToString()</code>



struct en C vs Clase en Java (Pregunta 16)

struct en C:

```
struct Punto {
    double x;
    double y;
};

// Función SUelta (no pertenece al struct)
double distanciaAlOrigen(struct Punto p) {
    return sqrt(p.x * p.x + p.y * p.y);
}
```

¿Qué le falta al `struct`?

Característica	<code>struct</code> en C	Clase en Java
Métodos integrados	✗ Funciones sueltas	☑ Métodos dentro de la clase
Encapsulamiento	✗ Todos los campos públicos	☑ <code>private/public</code>
Herencia	✗ No existe	☑ <code>extends</code>
Polimorfismo	✗ No existe	☑ Sobrecarga/sobreescritura
Constructores	✗ Inicialización manual	☑ <code>new Clase()</code>

💡 **Conclusión:** Un `struct` es solo un **contenedor de datos**. Una clase es un **objeto con identidad, estado y comportamiento**.

← BACK Emulando POO en C (Pregunta 17)

```
#include <math.h>
#include <stdio.h>

// 1. Definir el "objeto" como struct
typedef struct {
    double x;
    double y;
} Punto;

// 2. "Método" como función que recibe puntero al struct (¡this manual!)
double calculaDistanciaAOrigen(Punto *self) {
    return sqrt(self->x * self->x + self->y * self->y);
}

// 3. "Constructor" como función que inicializa y devuelve struct
Punto crearPunto(double x, double y) {
    Punto p;
    p.x = x;
    p.y = y;
    return p;
}

int main() {
    // "Instanciación"
    Punto p = crearPunto(3.0, 4.0);

    // "Llamada a método" (pasando this explícitamente)
    double d = calculaDistanciaAOrigen(&p);

    printf("Distancia: %.1f\n", d); // → 5.0
    return 0;
}
```

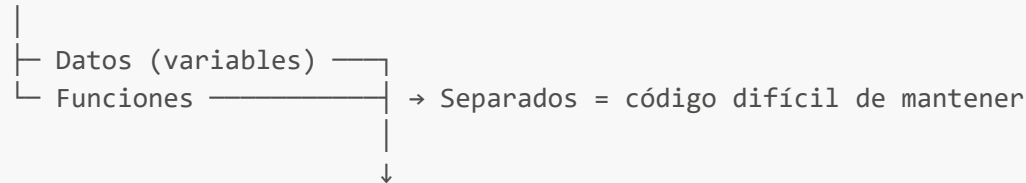
¿Qué pasó con `this`?

- En C **no existe** → lo emulamos pasando un puntero (`Punto *self`) como primer parámetro
- En C++/Java el compilador lo hace **automáticamente** detrás de escena

💡 **Historia:** C++ nació como "C con clases" precisamente para automatizar esto.

☑ Resumen visual: Evolución de paradigmas

Programación procedural (C)



Programación orientada a objetos (Java)

└ Clase = [Datos + Comportamiento] → Unidad coherente que modela el mundo real

🚀 ¿Qué sigue?

Ahora que dominas los fundamentos, los próximos pasos naturales son:

1. **Encapsulamiento avanzado:** `private`, getters/setters, `final`
2. **Herencia y polimorfismo:** `extends`, `@Override`, interfaces
3. **Constructores avanzados:** `this()` para delegar, bloques estáticos
4. **Paquetes y visibilidad:** `public/protected/default/private`